Virtual Memory Simulation Report
Author: Tristan Dyson; Date: October 27 2026

Report on a virtual memory simulator using clock replacement with fixed allocation and local replacement.

1 Testing Approach And Methodology

Inputted files allowed for whitespace (e.g. linebreaks or spaces) to separate words and references were case insensitive (i.e. “W” or “w” for a write operation). To test my parser properly read files regardless of whitespace I created a python script which modified a process file by randomly interspersing whitespace and modifying capitalisation (Appendix A contains generated samples). As only whitespace / capitalisation was changed, if the program output different results it would indicate a parsing bug. I then ran the virtual memory simulator on the original vs generated files and noted the generated outputs were identical (Appendix B) which indicated the parser was reliable.

To test the clock replacement policy I created a new sample using a single process. I worked out on paper the number of page faults and current pages in memory at each instruction. My Memory class was then modified to print out current pages in memory, allowing me to compare between the two.

2 Results Analysis And Observations

For the simulator a page fault resulted in the process being blocked for 4 units of time. If this required swapping out a dirty page the delay increased from 4 to 5 units of time. I expected more faults to raise a process’ turnaround time (as it is blocked and loses its time quantum) but barely affect system throughput since context switches had no overhead in this simulation, allowing the CPU to stay utilised while any process is unblocked. I tested by running the simulator using 5 processes, in one case each process had 8 frames each, while in the other each process only had 1 page frame (Appendix C). Fewer page frames per process resulted in more page faults, going from 9.6 to 14 faults on average, with average turnaround time going from 67.4 to 79.6 time units. Despite this both systems finished all processes in relatively similar times (112 to 113 time units) as theorized.

As swapping out a dirty page takes longer than a clean one, I expected that a process of all reads, when it had its operations changed to all writes would have an increased turn around time. I simulated this and the number of writebacks increased from 0 to 6, resulting in turnaround time increasing from 61 to 67 (Appendix D).

3 Edge Case Discussion And Algorithm Behaviour

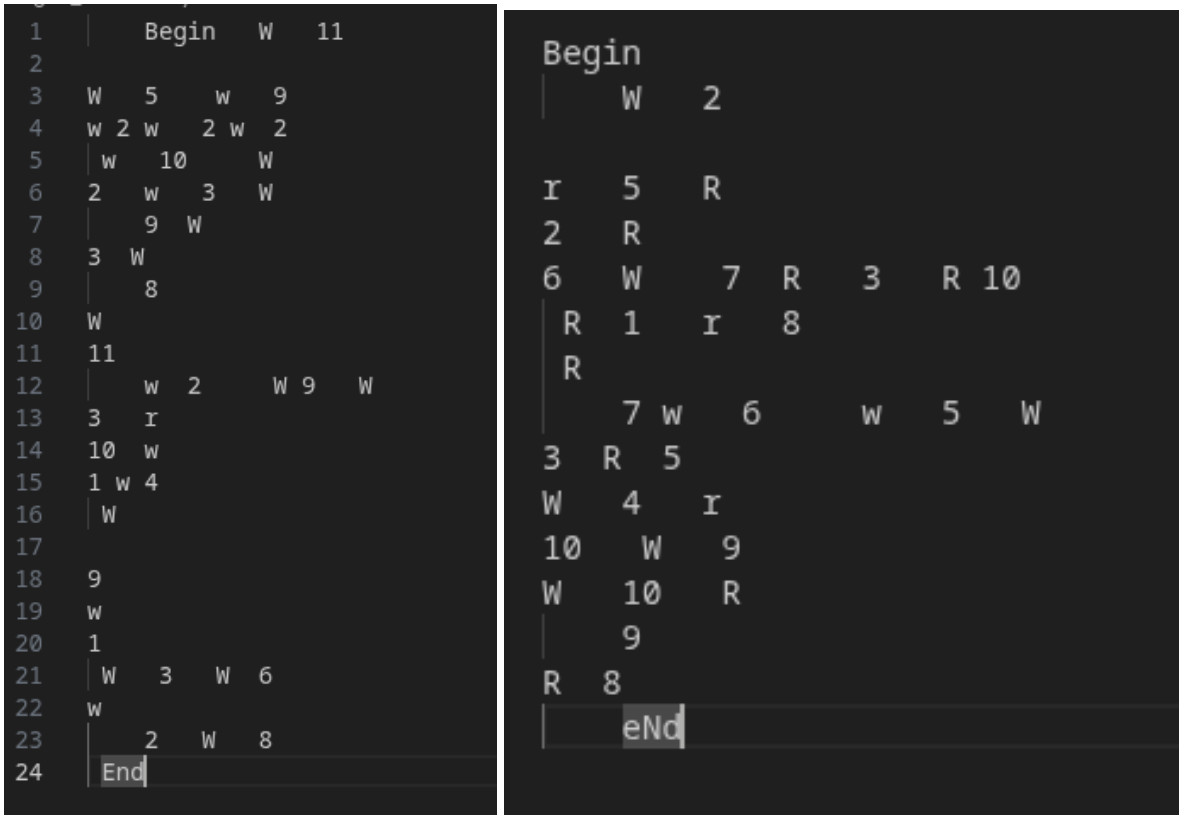
I handled edge cases with process file formats through my Python script as detailed in section 1 and processes that were all read / all write operations. I tested minimal and maximal inputs for page frames per process (1-50). For 50 page frames per process I expected each process to fault once per unique page accessed as none of my processes accessed more than 50 unique pages. I read through the number of faults for this process and my theory was the case, with each process faulting for the number of unique pages that process accessed.

4 Results Review & Debugging

Testing demonstrated turnaround time is negatively impacted by the number of faults and dirty page writebacks to secondary storage, while throughput barely changed.

During my debugging an issue I noticed was the number of faults recorded was occasionally one higher than expected. I reviewed that faults were created by my Memory class, which would optionally return a Fault object when an operation resulted in a block (i.e. memory access on a page that isn’t in memory). I realised this was semantically incorrect for releasing a process from memory, while this can block when writing back a dirty page to secondary memory this is not a page fault. To fix this I changed my Fault class to Block, with it containing metadata on the cause of the block. From there I could count the number of Block objects which were caused by a page fault to avoid this miscounting.

Appendix A: Whitespace Generation



```
    Begin   W   11

W   5    w   9
w 2 w   2 w  2
 w   10     W
2   w   3   W
    9  W
3  W
    8
W
11
    w  2     W 9   W
3   r
10  w
1 w 4
 W

9
w
1
 W   3   W  6
w
    2   W   8
 End
```

```
Begin
    W   2

r   5   R
2   R
6   W    7  R   3   R 10
 R  1   r   8
 R
    7 w   6     w   5   W
3  R  5
W   4   r
10   W   9
W   10   R
    9
R  8
    eNd
```

Example generated process files with randomly interspersed whitespace and variable capitalisation.

Appendix B: Program Runs

```
Clock - Fixed-Local Replacement:
Process Name          Turnaround Time # Faults # Write-Backs  Fault Times
generated                      71       12            6  {0, 5, 10, 15, 20, 27, 32, 38, 45, 53, 59, 64}
[tristan@archlinux A3]$ /usr/bin/python /home/tristan/Downloads/A3/generator.py
[tristan@archlinux A3]$ java A3 3 3 generated.txt
Clock - Fixed-Local Replacement:
Process Name          Turnaround Time # Faults # Write-Backs  Fault Times
generated                      71       12            6  {0, 5, 10, 15, 20, 27, 32, 38, 45, 53, 59, 64}
```

The program generates identical results regardless of whitespace.

Appendix C: Turnaround times with a different number of faults

```
[tristan@archlinux A3]$ java A3 40 3 TristTest1.txt TristTest2.txt TristT
est3.txt TristTest4.txt TristTest5.txt
Clock - Fixed-Local Replacement:
Process Name      Turnaround Time  # Faults  # Write-Backs  Fault Times
Process1          70               9         0              {0, 5, 11, 16, 22, 42, 47, 59, 65}
Process2          34               5         1              {0, 6, 12, 17, 23}
Process3          51               7         1              {0, 7, 13, 18, 24, 34, 45}
Process4          70               9         1              {0, 8, 14, 19, 26, 35, 43, 48, 60}
Process5          112              18        1              {0, 9, 15, 20, 27, 39, 46, 51, 60, 66, 71, 76, 81, 86, 91, 96, 101, 106}
[tristan@archlinux A3]$ java A3 5 3 TristTest1.txt TristTest2.txt TristTest3.txt TristTest4.txt TristTest5.txt
Clock - Fixed-Local Replacement:
Process Name      Turnaround Time  # Faults  # Write-Backs  Fault Times
Process1          113              21        0              {0, 5, 10, 15, 20, 26, 31, 37, 43, 49, 55, 61, 67, 72, 78, 83, 88, 93, 98, 103, 108}
Process2          40               7         3              {0, 6, 11, 16, 21, 27, 33}
Process3          65               11        5              {0, 7, 12, 17, 22, 28, 34, 40, 46, 52, 58}
Process4          77               13        5              {0, 8, 13, 18, 24, 29, 35, 41, 47, 53, 59, 65, 70}
Process5          103              18        1              {0, 9, 14, 19, 25, 30, 36, 42, 48, 54, 60, 66, 71, 77, 82, 87, 92, 97}
```

Appendix D: Turnaround times with read vs write only

```
[tristan@archlinux A3]$ java A3 5 3 WriteOnly.txt
Clock - Fixed-Local Replacement:
Process Name      Turnaround Time  # Faults  # Write-Backs  Fault Times
Process1          67               10        6              {0, 5, 11, 17, 22, 32, 38, 45, 54, 60}
[tristan@archlinux A3]$ java A3 5 3 ReadOnly.txt
Clock - Fixed-Local Replacement:
Process Name      Turnaround Time  # Faults  # Write-Backs  Fault Times
Process1          61               10        0              {0, 5, 11, 17, 22, 32, 37, 43, 51, 56}
```